

# Guía del Profesor Sintaxis y Semántica

Guía del Profesor — Tema 02

Matías Gel

Ciclo lectivo 2026

## Índice de Contenidos

|                                                                                              |          |
|----------------------------------------------------------------------------------------------|----------|
| <b>1. Guía del Profesor — Tema 02: Sintaxis y Semántica de Lenguajes</b>                     | <b>2</b> |
| 1.1. 1. Propósito de esta guía . . . . .                                                     | 2        |
| 1.2. 2. Estado de artefactos del tema . . . . .                                              | 2        |
| 1.3. 3. Plan de clase por bloques (120 min) . . . . .                                        | 3        |
| 1.3.1. Preparación (−10 min antes) . . . . .                                                 | 3        |
| 1.3.2. Apertura (5 min) — F-00, F-01 . . . . .                                               | 3        |
| 1.3.3. Bloque 1 — Sintaxis vs. semántica (20 min) — F-02 a F-06b . . . . .                   | 3        |
| 1.3.4. Bloque 2 — Estructura léxica (20 min) — F-07 a F-11 . . . . .                         | 3        |
| 1.3.5. Bloque 3 — Gramáticas formales, derivación y árboles (30 min) — F-12 a F-20 . . . . . | 4        |
| 1.3.6. Bloque 4 — Diagramas de sintaxis (10 min) — F-21 a F-23 . . . . .                     | 4        |
| 1.3.7. Bloque 5 — Síntesis de semántica (12 min) — F-24 a F-28 . . . . .                     | 5        |
| 1.3.8. Bloque 6 — Pipeline e IA generativa (15 min) — F-29 a F-34 . . . . .                  | 5        |
| 1.3.9. Cierre (8 min) — F-35, F-36, F-37 . . . . .                                           | 5        |
| 1.3.10. Bloque 2 (20 min): Léxico y tokenización . . . . .                                   | 5        |
| 1.3.11. Bloque 3 (30 min): Gramáticas formales, derivación y árboles . . . . .               | 6        |
| 1.3.12. Bloque 4 (10 min): Diagramas de sintaxis . . . . .                                   | 6        |
| 1.3.13. Bloque 5 (12 min): Síntesis de semántica . . . . .                                   | 6        |
| 1.3.14. Bloque 6 (15 min): Pipeline e IA . . . . .                                           | 6        |
| 1.3.15. Cierre (8 min) . . . . .                                                             | 6        |
| 1.4. 4. Núcleo conceptual que no puede faltar . . . . .                                      | 7        |
| 1.5. 5. Material fuente incorporado — contenido inline . . . . .                             | 7        |
| 1.5.1. 5.1 Definición de un lenguaje de programación . . . . .                               | 7        |
| 1.5.2. 5.2 Criterios sintácticos de un buen lenguaje . . . . .                               | 8        |
| 1.5.3. 5.3 Separación léxico/sintáctico — las tres razones (Sebesta §4.1) . . . . .          | 8        |
| 1.5.4. 5.4 Lexemas y tokens — tablas completas . . . . .                                     | 9        |
| 1.5.5. 5.5 Gramáticas, derivación y ambigüedad . . . . .                                     | 10       |
| 1.5.6. 5.6 Nombres, entorno y binding (puente a Tema 09) . . . . .                           | 12       |
| 1.5.7. 5.7 Clasificación de errores en TypeScript . . . . .                                  | 13       |
| 1.5.8. 5.8 Pipeline del compilador — diagrama inline (B6) . . . . .                          | 14       |
| 1.6. 6. Estrategias de mediación en aula . . . . .                                           | 14       |
| 1.6.1. 6.1 Preguntas de diagnóstico rápido . . . . .                                         | 14       |
| 1.6.2. 6.2 Respuestas a confusiones frecuentes . . . . .                                     | 15       |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 1.7. 7. Evaluación formativa — ejercicios en clase . . . . .                      | 15 |
| 1.7.1. Actividad B1 — Clasificación de errores TS (6 min, F-06 → F-06b) . . . . . | 15 |
| 1.7.2. Actividad B3 — derivación en pizarrón (5 min) . . . . .                    | 16 |
| 1.7.3. Cierre — ticket de salida (2 min) . . . . .                                | 16 |
| 1.8. 8. Riesgos de implementación y mitigaciones . . . . .                        | 16 |
| 1.9. 9. Límites de scope y continuidad curricular . . . . .                       | 17 |
| 1.10. 10. Checklist docente previo al dictado . . . . .                           | 17 |

## 1. Guía del Profesor — Tema 02: Sintaxis y Semántica de Lenguajes

**Materia:** Paradigmas y Lenguajes de Programación 2026 **Institución:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Docente:** Matías Gel **Tema:** 02 — Sintaxis y Semántica de Lenguajes **Duración:** 120 minutos · Semana 1 · Clase 2 de 2 **Perfil docente:** profesor-teórico

**Propósito de este documento:** Guía de dictado 100 % autocontenida. Todo el material fuente relevante (cátedra, Sebesta, Gabbrielli & Martini) está incorporado inline. No es necesario abrir ningún otro archivo durante la preparación ni el dictado.

### 1.1. 1. Propósito de esta guía

1. Entrar al aula con un hilo narrativo claro de punta a punta.
2. Tener a mano definiciones textuales, tablas, ejemplos y contraejemplos del material fuente, sin salir de este documento.
3. Identificar riesgos didácticos y sus intervenciones antes de que ocurran.
4. Evitar desalineaciones de scope respecto del diseño aprobado.

### 1.2. 2. Estado de artefactos del tema

| Artefacto                                      | Archivo                                        | Estado                |
|------------------------------------------------|------------------------------------------------|-----------------------|
| Diseño del tema (alcance, fuentes, decisiones) | <code>diseno.md</code>                         | APROBADO (2026-03-20) |
| Minuta de dictado minuto a minuto              | <code>minuta.md</code>                         | GENERADA              |
| Filminas F-00 a F-37                           | <code>filminas.md</code>                       | PRODUCIDAS            |
| Guía de estudio del alumno                     | <code>guia-estudio.md</code>                   | PRODUCIDA + PDF       |
| Trabajo práctico (quiz Moodle)                 | <code>tp.md</code> , <code>tp-quiz.gift</code> | PRODUCIDOS            |
| Reporte calidad escritura                      | <code>writing-report.md</code>                 | EJECUTADO             |

No se necesita abrir ninguno de estos archivos durante el dictado — esta guía los resume y extrae lo relevante.

### 1.3. 3. Plan de clase por bloques (120 min)

#### 1.3.1. Preparación (−10 min antes)

- Abrir [Deno Playground](#) — preparar el snippet de la actividad B1.

- Escribir en el pizarrón antes de que entren los alumnos:

```
sintaxis = forma
semántica = significado
```

- Verificar proyector con F-00.

#### 1.3.2. Apertura (5 min) — F-00, F-01

Pregunta disparadora (dejar leer, no leer en voz alta): > “¿Cómo sabe el compilador si tu programa está bien escrito?”

Esperar reacciones. Encuadrar: “Hoy abrimos la caja del compilador. En la clase anterior *tsc* compilaba *TypeScript* a *JavaScript* — hoy vemos cómo decide si lo que escribimos es válido.”

#### 1.3.3. Bloque 1 — Sintaxis vs. semántica (20 min) — F-02 a F-06b

**Objetivo:** distinguir forma/significado. Consolidar tres tipos de error.

| #  | Actividad                                | Tiempo | Filminas    |
|----|------------------------------------------|--------|-------------|
| 1a | Definir sintaxis con preguntas abiertas  | 4 min  | F-02, F-03  |
| 1b | Definir semántica — punto de tensión     | 4 min  | F-04        |
| 1c | Criterios sintácticos de un buen LP      | 4 min  | F-05        |
| 1d | Actividad parejas + cuadro de errores TS | 6 min  | F-06, F-06b |

**Riesgo:** el grupo reduce “semántica” a “tipos”. → **Intervención:** “Los tipos son semántica estática, pero no la única. Semántica es todo lo que da significado al programa — estático y dinámico.”

#### 1.3.4. Bloque 2 — Estructura léxica (20 min) — F-07 a F-11

**Objetivo:** del texto plano al token. Rol del lexer.

| #  | Actividad                                                          | Tiempo | Filminas |
|----|--------------------------------------------------------------------|--------|----------|
| 2a | Texto plano $\rightarrow$ ¿cuántas piezas?                         | 4 min  | F-07     |
| 2b | Tres razones para separar léxico/sintáctico                        | 3 min  | F-08     |
| 2c | Categorías léxicas de un LP                                        | 4 min  | F-09     |
| 2d | Lexema vs. token — tabla<br><code>indice = 5 * contador + 1</code> | 4 min  | F-10     |
| 2e | El lexer como componente del pipeline                              | 5 min  | F-11     |

**Nota pedagógica:** se anticipa el Cap. 4 de Sebesta (lexer) antes del Cap. 3 (gramáticas). Es intencionalmente inverso al libro: *concreto*  $\rightarrow$  *abstracto*. Si un alumno pregunta, confirmarlo: “*Lo hacemos así porque ver primero el lexer hace que las gramáticas tengan anclaje práctico.*”

### 1.3.5. Bloque 3 — Gramáticas formales, derivación y árboles (30 min) — F-12 a F-20

**Objetivo:** lectura funcional de BNF/EBNF, derivación manual, detección de ambigüedad.

| #  | Actividad                                            | Tiempo | Filminas   |
|----|------------------------------------------------------|--------|------------|
| 3a | Tupla de gramática libre de contexto                 | 5 min  | F-12, F-13 |
| 3b | BNF: metasímbolos, LHS/RHS                           | 5 min  | F-14, F-15 |
| 3c | Derivación completa de $A := B * (A + C)$            | 8 min  | F-16       |
| 3d | Árbol sintáctico — misma derivación                  | 5 min  | F-17       |
| 3e | Ambigüedad: $J := 1 + 2 * 3$ — dos árboles           | 5 min  | F-18       |
| 3f | EBNF — extensiones <code>[]</code> , <code>{}</code> | 2 min  | F-19, F-20 |

**Punto de control al cerrar:** pedir a un alumno que explique por qué dos árboles implican dos significados distintos antes de avanzar.

### 1.3.6. Bloque 4 — Diagramas de sintaxis (10 min) — F-21 a F-23

**Objetivo:** lectura gráfica y equivalencia con reglas BNF.

- Railroad diagram para condicional en lenguaje simple.
- Conexión con documentación de lenguajes reales (TypeScript Handbook, MDN).
- Énfasis: son otra forma de expresar la misma gramática — no una alternativa conceptualmente distinta.

**1.3.7. Bloque 5 — Síntesis de semántica (12 min) — F-24 a F-28**

**Objetivo:** panorámica de semántica sin formalismo pesado.

- Semántica estática: gramáticas de atributos, chequeo de tipos en TypeScript.
- Semántica dinámica: operacional, denotacional, axiomática — nivel introductorio.
- Mención breve: nombres / entorno / binding (puente hacia Tema 09).

**Criterio de alcance:** NO derivar formalismos completos en pizarrón. Reservar profundidad para temas posteriores.

**1.3.8. Bloque 6 — Pipeline e IA generativa (15 min) — F-29 a F-34**

**Objetivo:** cerrar relevancia contemporánea.

- Pipeline Lexer → Parser → Type-checker → Emitter (TypeScript/`tsc`).
- Constrained decoding como aplicación actual de gramáticas formales.
- Encuadre: la IA no elimina la necesidad de entender sintaxis — la presupone.

**1.3.9. Cierre (8 min) — F-35, F-36, F-37**

- Mapa conceptual integrador.
- Tres preguntas de chequeo rápido.
- Ticket de salida: *“Un concepto claro y uno que quiero repasar.”*
- Avance Tema 03: sistema de tipos como profundización de semántica estática.
- Cuadro de tres errores (sintáctico, semántico estático, dinámico).
- Actividad breve en parejas (clasificación de ejemplos TS).
- Filminas: F-02 a F-06b.

Riesgo didáctico:

- Que el grupo reduzca “semántica” solo a “tipos”.

Intervención:

- Enfatizar que tipos son una parte de la semántica estática, no toda la semántica.

**1.3.10. Bloque 2 (20 min): Léxico y tokenización**

**Objetivo:** explicar la transición texto->token y el rol del lexer.

- Lexema vs token con ejemplo clásico `indice = 5 * contador + 1`.
- Separación léxico/sintáctico por simplicidad, eficiencia y portabilidad.
- Filminas: F-07 a F-11.

Decisión pedagógica explícita:

- Se anticipa material de análisis léxico antes de gramáticas para anclaje concreto.

### 1.3.11. Bloque 3 (30 min): Gramáticas formales, derivación y árboles

Objetivo: lectura funcional de BNF/EBNF y detección de ambigüedad.

- Tupla de gramática libre de contexto (uso liviano, no formalismo duro).
- Derivación de  $A := B * (A + C)$ .
- Árbol sintáctico y ambigüedad con  $1 + 2 * 3$ .
- Filminas: F-12 a F-20.

Punto de control:

- Antes de cerrar bloque, pedir que expliquen por qué dos árboles implican dos significados.

### 1.3.12. Bloque 4 (10 min): Diagramas de sintaxis

Objetivo: lectura gráfica y equivalencia con reglas.

- Railroad diagrams para condicional.
- Conexión con documentación de lenguajes reales.
- Filminas: F-21 a F-23.

### 1.3.13. Bloque 5 (12 min): Síntesis de semántica

Objetivo: panorámica clara sin salir del scope.

- Semántica estática: gramáticas de atributos, chequeo de tipos.
- Semántica dinámica: operacional, denotacional, axiomática (nivel introductorio).
- Filminas: F-24 a F-28.

Criterio de alcance:

- No derivar formalismos completos en pizarrón. Reservar profundidad para temas posteriores.

### 1.3.14. Bloque 6 (15 min): Pipeline e IA

Objetivo: cerrar relevancia contemporánea.

- Pipeline compilador/intérprete.
- Constrained decoding como aplicación actual de gramáticas.
- Filminas: F-29 a F-34.

### 1.3.15. Cierre (8 min)

Objetivo: consolidación y transición.

- Mapa conceptual final.
- Preguntas de chequeo rápido.
- Filminas: F-35, F-36, F-37.

---

#### 1.4. 4. Núcleo conceptual que no puede faltar

1. **Sintaxis determina forma; semántica determina significado.**
  2. **Lexer y parser resuelven capas distintas** — no compiten, se complementan secuencialmente.
  3. **BNF/EBNF no son “solo teoría”** — definen estructuras que compiladores y herramientas procesan.
  4. **Ambigüedad gramatical es un defecto de diseño** porque rompe la interpretación única del programa.
  5. **Semántica estática y dinámica ocurren en momentos distintos** del ciclo de vida del programa.
- 

#### 1.5. 5. Material fuente incorporado — contenido inline

Todo el contenido que sigue está extraído y condensado de las fuentes primarias. No es necesario abrir ningún archivo externo durante la preparación ni el dictado.

---

##### 1.5.1. 5.1 Definición de un lenguaje de programación

**Fuente:** Filminas de cátedra UNTDF 2025

*“Un lenguaje de programación es una notación formal para describir algoritmos a ser ejecutados por computadoras.”*

##### Componentes:

| Componente       | Qué define                                                    |
|------------------|---------------------------------------------------------------|
| <b>Sintaxis</b>  | Forma de las expresiones, sentencias y unidades de programa   |
| <b>Semántica</b> | Significado de expresiones, sentencias y unidades de programa |

---

La definición de un lenguaje permite determinar: (a) si un programa es válido, y (b) cuál es su significado o efecto.

### Ejemplo canónico — sentencia condicional en C:

- **Sintaxis:** `if (<expresión>) <sentencia>`
- **Semántica:** “Si el valor actual de la expresión es cierto, se ejecuta la sentencia siguiente”

**Uso en clase:** abrir B1 y legitimar el vocabulario técnico antes de entrar en TypeScript.

### 1.5.2. 5.2 Criterios sintácticos de un buen lenguaje

**Fuente:** Filminas de cátedra UNTDF 2025

| Criterio                         | Descripción                                              | Ejemplo                                                |
|----------------------------------|----------------------------------------------------------|--------------------------------------------------------|
| <b>Legibilidad</b>               | El código debe ser fácil de leer y entender              | Python prioriza esto explícitamente                    |
| <b>Facilidad de escritura</b>    | Sintaxis compacta y expresiva                            | Perl sacrificó legibilidad por expresividad comprimida |
| <b>Facilidad de verificación</b> | Debe ser posible razonar sobre corrección                | Tipado estático facilita esto                          |
| <b>Facilidad de traducción</b>   | El compilador/intérprete puede procesarla eficientemente | Gramáticas no ambiguas son requisito                   |
| <b>Carencia de ambigüedad</b>    | Cada sentencia tiene exactamente una interpretación      | Ver §5.5                                               |

**Elementos léxicos de un LP:** Caracteres · Identificadores · Operadores · Palabras clave y reservadas · Comentarios · Separadores

**Reglas léxicas:** definen el alfabeto del lenguaje y cómo combinar caracteres para formar palabras válidas. (Java y Python tratan diferente las mayúsculas — regla léxica.)

**Reglas sintácticas:** definen cómo pueden formarse las sentencias a partir de palabras.

### 1.5.3. 5.3 Separación léxico/sintáctico — las tres razones (Sebesta §4.1)

**Fuente:** Sebesta, *Concepts of Programming Languages* 12<sup>a</sup> ed., Cap. 4



| Razón                   | Texto literal (Sebesta)                                                                                                                                                                                                                                                                                         | Traducción para clase                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 1.<br><b>Simplicity</b> | <i>“Techniques for lexical analysis are less complex than those required for syntax analysis, so the lexical-analysis process can be simpler if it is separate. Also, removing the low-level details of lexical analysis from the syntax analyzer makes the syntax analyzer both smaller and less complex.”</i> | Separar ortografía de gramática simplifica ambos analizadores         |
| 2.<br><b>Efficiency</b> | <i>“Although it pays to optimize the lexical analyzer, because lexical analysis requires a significant portion of total compilation time, it is not fruitful to optimize the syntax analyzer. Separation facilitates this selective optimization.”</i>                                                          | El lexer corre millones de veces — vale optimizarlo de forma aislada  |
| 3. <b>Portability</b>   | <i>“Because the lexical analyzer reads input program files and often includes buffering of that input, it is somewhat platform dependent. However, the syntax analyzer can be platform independent. It is always good to isolate machine-dependent parts of any software system.”</i>                           | Separación de responsabilidades — principio de ingeniería de software |

**Cita clave sobre el rol del lexer (Sebesta §4.2):** > *“A lexical analyzer is essentially a pattern matcher. An input program appears to a compiler as a single string of characters. The lexical analyzer collects characters into logical groupings and assigns internal codes to the groupings according to their structure.”*

**Lo que hace el lexer además de tokenizar:** - Salta comentarios y espacios en blanco (no relevantes para el significado). - Inserta lexemas de nombres de usuario en la tabla de símbolos. - Detecta errores sintácticos en tokens (p.ej. literales de punto flotante malformados).

#### 1.5.4. 5.4 Lexemas y tokens — tablas completas

**Fuente:** Sebesta §4.2 + filminas cátedra UNTDF 2025

**Ejemplo Sebesta** — sentencia de asignación en C:

```
result = oldsum - value / 100;
```

| Token     | Lexema |
|-----------|--------|
| IDENT     | result |
| ASSIGN_OP | =      |
| IDENT     | oldsum |
| SUB_OP    | -      |
| IDENT     | value  |
| DIV_OP    | /      |
| INT_LIT   | 100    |
| SEMICOLON | ;      |

**Ejemplo cátedra UNTDF** — expresión aritmética con identificadores en español:

```
indice = 5 * contador + 1
```

| Token     | Lexema   |
|-----------|----------|
| IDENT     | indice   |
| ASSIGN_OP | =        |
| INT_LIT   | 5        |
| MULT_OP   | *        |
| IDENT     | contador |
| ADD_OP    | +        |
| INT_LIT   | 1        |

**Principio del longest substring (Sebesta):** al tokenizar, se elige siempre el token más largo posible. Así `doif` es un identificador (no `do + if`), y `x12` es un identificador (no `x + 12`).

#### 1.5.5. 5.5 Gramáticas, derivación y ambigüedad

**Fuente:** Louden & Lambert, Cap. 6 §§6.2–6.4

**1.5.5.1. BNF — estructura de una gramática libre de contexto** Una gramática  $G = (VN, VT, P, S)$  donde: -  $VN$  = símbolos no-terminales (categorías, entre  $\langle \rangle$ ) -  $VT$  = símbolos terminales (tokens del lexer) -  $P$  = producciones ( $A \rightarrow$  ) -  $S$  = símbolo inicial

#### 1.5.5.2. Derivación completa — ejemplo de clase B3

$\langle \text{Sentencia} \rangle \quad \langle \text{Identificador} \rangle := \langle \text{Expresion} \rangle$

```

A := <Expresion>
A := <Expresion> * <Factor>
A := <Factor> * <Factor>
A := <Identificador> * <Factor>
A := B * <Factor>
A := B * ( <Expresion> )
A := B * ( <Expresion> + <Termino> )
A := B * ( A + C )

```

### 1.5.5.3. Árbol sintáctico — misma derivación

```

      <Sentencia>
      /      \
A  :=      <Expresion>
           /      \
        <Expresion> * <Factor>
           |          |
        <Factor>      ( <Expresion> )
           |          /      \
        <Identificador> <Expresion> + <Termino>
           |          |          |
           B          <Factor>    <Identificador>
                   |          |
                   <Identificador> C
                   |
                   A

```

### 1.5.5.4. Ambigüedad — el caso canónico $3 + 4 * 5$ La gramática ingenua (ambigua):

```

<expr> → <expr> + <expr>
      | <expr> * <expr>
      | ( <expr> )
      | number

```

Para  $3 + 4 * 5$  esta gramática genera **dos árboles válidos**:

**Árbol 1** — multiplicación primero (resultado =  $3 + 20 = 23$ , matemáticamente correcto):

```

      +
      / \
3      *

```

$$\begin{array}{cc} / & \backslash \\ 4 & 5 \end{array}$$

**\*\*Árbol 2 — suma primero (resultado =  $(3+4)*5 = 35$ , matemáticamente incorrecto):\*\***

$$\begin{array}{cc} & * \\ & / \backslash \\ + & 5 \\ / \backslash \\ 3 & 4 \end{array}$$

**Cita Louden & Lambert (textual):** > “A grammar such as this, for which two distinct parse (or syntax) trees are possible for the same string, is ambiguous. Ambiguous grammars present difficulties, since no clear structure is expressed. To be useful, either the grammar must be revised to remove the ambiguity or a disambiguating rule must be stated to establish which structure is meant.”

**Solución — gramática no ambigua con precedencia en cascada:**

```
<expr>  → <expr> + <term> | <term>
<term>  → <term> * <factor> | <factor>
<factor> → ( <expr> ) | number
```

Ahora \* solo puede aparecer en posición más baja del árbol → tiene mayor precedencia que +.

**Intervención docente para B3:** “Si hay dos árboles válidos, hay dos posibles significados. En compilación eso es inadmisibile sin una regla extra — el compilador no puede adivinar.”

### 1.5.6. 5.6 Nombres, entorno y binding (puente a Tema 09)

**Fuente:** Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* (2023), Cap. 4 §4.1

*“A name is therefore a sequence of characters used to represent, or denote, another object.”*

**Distinción nombre vs. objeto (G&M §4.1):** > “Even though it might seem obvious, it is important to emphasize that a name and the object it denotes are not the same thing. A name, indeed, is just a character string, while its denotation can be a complex object such as a variable, a function, a type, and so on.”

Consecuencias para mencionar brevemente: - **Aliasing:** un objeto puede tener más de un nombre.  
- **Rebinding:** un mismo nombre puede denotar distintos objetos en distintos momentos.

**El entorno (G&M §4.1):** > “We will use the term *environment* to refer to that part of the implementation responsible for the associations between names and the objects that they denote.”

**Ejemplo mínimo para B5:**

```
let fie: number;      // nombre "fie" → variable (ubicación de memoria)
fie = 2;              // valor 2 se almacena en esa ubicación
```

→ “El compilador no trabaja con direcciones de memoria — trabaja con nombres. El entorno resuelve esa asociación.”

**Tipos de binding (mencionarlos sin profundizar):**

| Tipo            | Resolución               | Ejemplo                        |
|-----------------|--------------------------|--------------------------------|
| <b>Estático</b> | En tiempo de compilación | TypeScript, C++                |
| <b>Dinámico</b> | En tiempo de ejecución   | Algunos lenguajes de scripting |

**Recordatorio:** el tratamiento completo de entorno, scope rules y binding está diferido a **Tema 09**. No extenderse aquí.

### 1.5.7. 5.7 Clasificación de errores en TypeScript

**Fuente:** minuta de clase + ejemplos TypeScript verificados

| Código TypeScript                   | Tipo de error             | ¿Cuándo?    | ¿Quién detecta?                        |
|-------------------------------------|---------------------------|-------------|----------------------------------------|
| if true { }                         | <b>Sintáctico</b>         | Parsing     | Parser (el AST no llega a construirse) |
| const x: number = "hola"            | <b>Semántico estático</b> | Compilación | Type-checker                           |
| let y: any = undefined;<br>y.length | <b>Semántico dinámico</b> | Runtime     | Motor de ejecución                     |

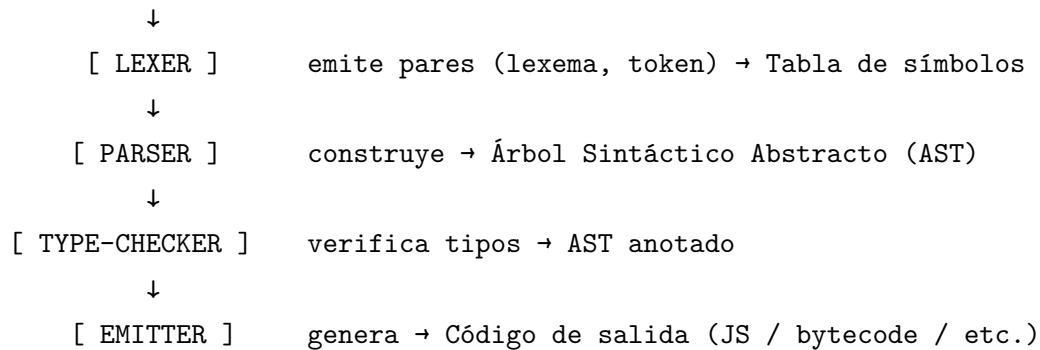
**Explicaciones para el docente:**

- `if true { }` → falta el paréntesis. El parser no puede construir el árbol porque la regla BNF del `if` requiere ( <expresión> ). El lexer tokenizó bien — el error está en la sintaxis.
- `const x: number = "hola"` → árbol sintáctico completamente válido. El error aparece cuando el type-checker asigna tipos a los nodos y detecta incompatibilidad. La sintaxis está bien; la semántica estática falla.

- `y.length` sobre `undefined` → el compilador no detecta el problema con `any` en compile-time. El error ocurre solo al ejecutar. La semántica dinámica falla.

### 1.5.8. 5.8 Pipeline del compilador — diagrama inline (B6)

Código fuente (texto plano)



**En TypeScript (tsc):** los cuatro componentes son un pipeline integrado. El lexer convierte `.ts` en stream de tokens; el parser construye el AST; el type-checker lo anota; el emitter produce `.js`.

**Constrained decoding — conexión con IA generativa (B6):** Las gramáticas BNF se usan hoy en sistemas de IA generativa para garantizar que la salida del modelo sea sintácticamente válida. El decodificador solo permite continuaciones de tokens que corresponden a una derivación válida de la gramática — exactamente los árboles de derivación de B3 en tiempo real.

## 1.6. 6. Estrategias de mediación en aula

### 1.6.1. 6.1 Preguntas de diagnóstico rápido

| Momento  | Pregunta                                                                            | Respuesta esperada                          |
|----------|-------------------------------------------------------------------------------------|---------------------------------------------|
| Apertura | <i>“¿Cómo sabe el compilador si tu programa está bien escrito?”</i>                 | “Por las reglas / la gramática”             |
| B1       | <i>“Un programa que compila, ¿siempre está correcto en runtime?”</i>                | <b>No</b> — puede fallar con error dinámico |
| B1       | <i>“¿Dónde detecta el compilador un error de paréntesis en un <code>if</code>?”</i> | En el parser, antes del type-checker        |

| Momento | Pregunta                                                                          | Respuesta esperada                                        |
|---------|-----------------------------------------------------------------------------------|-----------------------------------------------------------|
| B2      | “¿Qué produce como salida el lexer?”                                              | Pares (lexema, token)                                     |
| B3      | “¿Por qué dos árboles distintos para la misma cadena son un problema?”            | Implican dos significados → el compilador no puede elegir |
| Cierre  | “¿En qué momento del pipeline se detecta <code>const x: number = 'hola'?</code> ” | Type-checker (semántica estática)                         |

### 1.6.2. 6.2 Respuestas a confusiones frecuentes

**Confusión 1:** “Semántica = ejecución solamente” → “La semántica tiene parte estática (tipos se verifican antes de ejecutar) y parte dinámica. Tipos son semántica estática porque dan significado antes de ejecutar.”

**Confusión 2:** “BNF es memorizar símbolos raros” → “BNF es una forma de especificar estructura de manera que una herramienta la pueda procesar sin ambigüedad. TypeScript Handbook usa diagramas de sintaxis que son BNF visual. `tsc` tiene una BNF interna de TypeScript.”

**Confusión 3:** “La ambigüedad es un problema menor” → “Si hay dos árboles válidos,  $3 + 4 * 5$  da 23 o 35 según cuál árbol usemos. En compilación eso es inadmisibile sin una regla explícita.”

**Confusión 4:** “¿Para qué aprender BNF si el compilador ya lo hace solo?” → “El compilador lo hace solo porque alguien escribió la BNF. Los generadores de parsers (`lex`, `yacc`, `ANTLR`) toman una BNF como entrada y generan el analizador automáticamente.”

## 1.7. 7. Evaluación formativa — ejercicios en clase

### 1.7.1. Actividad B1 — Clasificación de errores TS (6 min, F-06 → F-06b)

**Formato:** parejas, 2 min de trabajo + 4 min de discusión.

```
// 1. ¿Qué tipo de error?
if true {
  console.log("hola")
}
```

```
// 2. ¿Qué tipo de error?
const x: number = "hola"

// 3. ¿Qué tipo de error?
let y: any = undefined
console.log(y.length)

// 4. ¿Es un error?
const z: number = 42
console.log(z.toString())
```

**Respuestas:** (1) sintáctico, (2) semántico estático, (3) semántico dinámico, (4) no es error.

### 1.7.2. Actividad B3 — derivación en pizarrón (5 min)

Pedir que deriven  $x + y * z$  con la gramática no ambigua:

```
<expr>  → <expr> + <term> | <term>
<term>  → <term> * <factor> | <factor>
<factor> → ( <expr> ) | IDENT
```

Pregunta clave: “¿Pueden derivar esta expresión de dos formas que den árboles distintos?” (Respuesta: **no** — esta gramática no es ambigua.)

### 1.7.3. Cierre — ticket de salida (2 min)

*“En papel: (a) un concepto que te quedó claro hoy, (b) uno que querés repasar. Dejalo en la mesa al salir.”*

Revisar en la próxima clase: si la mayoría menciona “ambigüedad” o “semántica dinámica” como pendiente, abrir el Tema 03 con una pregunta de repaso.

## 1.8. 8. Riesgos de implementación y mitigaciones

| Riesgo                                     | Probabilidad | Mitigación                                                |
|--------------------------------------------|--------------|-----------------------------------------------------------|
| Sobrecargar formalismo matemático temprano | Alta         | Ejemplos concretos en TS primero; formalizar solo después |



| Riesgo                                                | Probabilidad | Mitigación                                                               |
|-------------------------------------------------------|--------------|--------------------------------------------------------------------------|
| Perder tiempo en debate de herramientas IA            | Media        | Encapsular en B6 como aplicación, no como eje teórico                    |
| Desbordar scope hacia teoría completa de compiladores | Alta         | Repetir: <i>“introductorio hoy, profundidad en espacios específicos”</i> |
| Tiempo insuficiente en B3                             | Media        | Comprimir B4 (diagramas) — bloque más prescindible si hay que recortar   |
| Preguntas sobre ámbito/scope en B5                    | Media        | <i>“Exactamente Tema 09. Por ahora, solo la intuición.”</i>              |

### 1.9. 9. Límites de scope y continuidad curricular

**Fuera de scope hoy:** - Implementación de algoritmos LL/LR (parsers concretos). - Desarrollo formal completo de semántica operacional/denotacional/axiomática. - Scope rules y binding en detalle → **Tema 09: Variables, Binding y Ámbito.** - Construcción de un lexer desde cero.

**Continuidad:** - **Tema 03:** sistema de tipos como profundización de semántica estática. - **Tema 09:** entorno, binding, scope — G&M Cap. 4 completo.

### 1.10. 10. Checklist docente previo al dictado

- ☐ Revisé timing de cada bloque (suma total: 120 min).
- ☐ Deno Playground abierto con snippet de la actividad B1.
- ☐ Tengo claro qué parte del pipeline es léxica, cuál sintáctica, cuál semántica.
- ☐ Preparé la pregunta de control de ambigüedad (cierre de B3).
- ☐ Sé cuándo comprimir si B3 se extiende (recortar B4).
- ☐ No salgo del scope del diseño aprobado en ningún bloque.
- ☐ Tengo en mente los tres errores TS del cuadro §5.7.

Con este checklist en verde, el tema queda listo para dictado consistente, trazable y autocontenido.